

Relazione Progetto Finale

Andrea Scotto - S6463898

Titolo Progetto: Biliardo 3D

Scopo del progetto:

Lo scopo di questo progetto è di implementare un biliardo 3D funzionante quanto più realistico possibile. Le principali feature che avevo in mente prima dell'inizio del progetto sono, in ordine di importanza: movimento delle biglie con velocità calibrabile dall'utente tramite drag della stecca, urti semi-realistici delle biglie tra di loro e con le sponde, decelerazione delle biglie graduale secondo un coefficiente d'attrito fisso del tavolo, regole classiche del biliardo con assegnazione di biglie intere e spezzate, spin delle biglie che ne modifica sia la rotazione a vista sia il loro comportamento durante gli urti.

Tappa 1

L'obiettivo della tappa 1 era semplicemente effettuare un primo setup della finestra con contesto OpenGL e caricamento di glad e la stesura del file CMakeLists.txt usando come esempio quelli provenienti dai laboratori eseguiti durante l'anno. Una volta eseguito ciò, sempre partendo dai risultati ottenuti dal laboratorio 7, ho eseguito un primo semplice rendering di una sfera senza shading. Successivamente ho aggiunto alla scena una Camera con rotazione definita tramite trackball per verificare la correttezza del rendering.

Tappa 2

L'obiettivo della tappa 2 era quello di introdurre un primo tipo di shading al progetto. Per la resa grafica che ha sugli oggetti tondeggianti ho preferito il phong shading. Prima di inserire direttamente il phong shading ho deciso di utilizzare lo stesso approccio del Laboratorio 6, cioè uno shading con colori fittizi per verificare il corretto comportamento

delle normali. Una volta verificato il corretto funzionamento sono andato a introdurre il phong shading, il quale ha introdotto una resa grafica nettamente migliore. Una volta introdotto lo shading, in vista della struttura del biliardo, ho voluto creare una distinzione di materiali tra sfere, nettamente più lucide, e panno del tavolo, chiaramente più opaco. Di conseguenza ho introdotto una struct Material, che contiene tutte le informazioni del materiale che avevo inizialmente collocato nella classe Lights, e un metodo ausiliario draw_mesh() che prende in input anche il materiale e di conseguenza renderizza una determinata mesh con un materiale specifico. Per controllare il corretto funzionamento di questo nuovo metodo ho introdotto due tipi di materiali nella classe Scene e ho creato una semplice mesh rappresentante il panno del biliardo.

Tappa 3

L'obiettivo della tappa 3 era quello di completare la struttura base del biliardo, provvisoriamente senza buche, definito come una serie di cubi schiacciati utilizzando la mesh proveniente dal Laboratorio 7. Una volta costruiti panno e sponde ho notato che, in quanto tutti gli oggetti all'interno della scena utilizzavano il phong shading, i bordi delle sponde apparivano tondeggianti, quindi ho deciso di implementare l'estensione proposta nel Laboratorio 7: shading diversi in base all'oggetto. Seguendo i suggerimenti, partendo dalla base contenuta nel file hotshaders.hh, ho utilizzato il tipo std::map per associare ad ogni nome di uniform la relativa location e ho ridefinito l'oggetto Shaders come una map "nome-shader" - programma e un puntatore allo shader attuale. Ho spostato tutta la logica inerente alle location delle uniform all'interno della classe Shaders. Nel momento in cui si vuole introdurre un nuovo shader si utilizza il nuovo metodo add, il quale chiama il vecchio metodo load passando i file shader corrispondenti. Ho modificato il metodo compile_attach_link in modo che scriva in una variabile l'id del programma creato in modo che possa essere inserito all'interno della mappa e successivamente la mappa locations viene popolata dalla chiamata di fetch_locs(), metodo privato che, per ogni uniform passata al fragment shader, ne ottiene la location. Adesso, le classi che prima scrivevano direttamente il valore della uniform, delegano questo compito alla classe Shaders chiamando il metodo set(). In questo modo ho

potuto utilizzare il phong shading sulla biglia e il flat sulla struttura del tavolo, in modo tale da eliminare l'effetto tondeggiante delle sponde. Successivamente ho completato la struttura del tavolo definendo il corpo sottostante il panno e le gambe del tavolo, sempre come cubi scalati opportunamente mediante le misure contenute nel nuovo namespace game. Come ultima modifica minimale, ho modificato la visuale iniziale per averla sul lato corto del tavolo e inclinata rispetto all'asse delle x, per avere già una visuale sul tavolo.

Tappa 4

L'obiettivo della tappa 4 era l'implementazione del movimento delle biglie e della loro interazione con le sponde, provvisoriamente usando solo una biglia nonostante i metodi siano stati scritti per il caso con più biglie. È stata modificata la logica di rendering in quanto adesso, per disegnare la sfera al centro del tavolo, non è più la classe Scene a decidere dove posizionarla, ma riceve la posizione dalla nuova classe Physics, la quale contiene al suo interno tutte le biglie attive (cioè non imbucate). Le biglie sono definite da una struct Ball contenente vettori posizione e velocità (inizializzata a un valore arbitrario per verificare il corretto funzionamento dell'implementazione della tappa). La classe Physics, inoltre, possiede un metodo chiamato all'interno del main loop update che, ricevendo un intervallo di tempo ricevuto dal clock aggiunto nel main, calcola la nuova posizione della biglia in base alla velocità e riduce quest'ultima seguendo un coefficiente d'attrito definito nel namespace game. Il secondo metodo della classe gestisce la collisione con le sponde confrontando la posizione del centro della biglia con la distanza minima che esso può raggiungere dalla sponda, se è minore del raggio della biglia quest'ultima viene riposizionata attaccata alla sponda appena colpita e la relativa componente della velocità viene riflessa.

Tappa 5

L'obiettivo della tappa 5 era il corretto posizionamento delle biglie all'inizio del gioco. Le posizioni delle biglie vengono calcolate nel metodo setup_rack. Il triangolo parte da tre quarti del tavolo e prende questo punto come punto iniziale in base al quale calcolare la posizione delle altre biglie. La componente x viene calcolata in base al triangolo che 3

biglie formano. Questo triangolo ha lato $2r$, di conseguenza l'altezza del triangolo è di $\sqrt{3} r$. La componente z calcola la distanza tra due biglie, cioè $2r$, mentre il coefficiente per il quale viene moltiplicato serve per calcolare gli offset dal centro. Successivamente ho deciso di colorare le biglie coerentemente con i colori tradizionali, definendo un array di coefficienti diffusi all'interno della Scene e non usando più un materiale fisso per le biglie. In questo modo ricavo il coefficiente dal numero di biglia, anch'esso aggiunto in questa tappa. Il metodo usato è `get_material`, che contiene due casi speciali: biglia bianca e nera, le altre biglie sono state colorate coerentemente all'array. In quanto la biglia nera non era posizionata al centro del rack ho deciso di spostare la biglia nera con quella centrale col metodo della libreria `standard swap`. Successivamente, per disegnare le strisce sulle biglie mezze ho modificato il `vertex_shader` facendogli dare in output anche le coordinate oggetto, ricevute dal `fragment_shader` che decide come colorare le biglie mezze, tramite la nuova uniform `isStriped`, a seconda del punto sull'asse y , modificando opportunamente coefficiente diffuso e ambiente.

Tappa 6

L'obiettivo della tappa era di implementare il corretto funzionamento degli urti fra biglie.

Per ogni coppia di biglie si verifica se la distanza tra i loro centri è minore di un diametro, in caso affermativo si calcola la nuova posizione. Viene fatto un ulteriore controllo prima di cambiare pure le velocità in quanto le biglie potrebbero essere già in allontanamento, se è così la velocità attuale è quella corretta e non viene modificata, altrimenti viene calcolata rispetto al vettore che congiunge i centri delle due biglie.

L'orientation di ogni biglia genera visivamente l'effetto di spin in base alla direzione nella quale la biglia si sta muovendo (lo spin non agisce sugli urti e sulle traiettorie). Ogni volta che la nuova posizione viene calcolata, viene aggiornato anche questo valore. La modifica è in base all'asse perpendicolare rispetto al vettore velocità della biglia e all'angolo del rotolamento calcolato come spostamento/raggio della biglia. La nuova orientation quindi viene applicata come composizione della vecchia con la rotazione dell'angolo ottenuto rispetto all'asse ottenuto.

Tappa 7

L'obiettivo era quello di implementare un sistema composto da diverse visuali in base al momento della partita. Ho deciso di tenere la visuale che avevo utilizzato finora come modalità FREE che non viene sfruttata nelle meccaniche di gioco. Ho definito una modalità AIM con una object distance minore in modo che si abbia un primo piano sulla biglia bianca per decidere dove colpirla. Infine ho implementato una vista TOP utilizzata quando le biglie sono in movimento. Successivamente ho associato un bottone per ogni vista: A per AIM, T per TOP, F per FREE tramite delle handle. AIM e TOP non sono definite con quaternioni come FREE in quanto presentano dei vincoli nella rotazione: in modalità TOP si può solo girare attorno all'asse delle y mentre in modalità AIM si può ruotare sia intorno all'asse y sia intorno all'asse x avendo però un vincolo su quest'ultimo per non avere una visuale da sotto il tavolo. Per le modalità TOP e AIM in view projection è calcolata differentemente la vista in base ai parametri delle singole visuali.

Tappa 8

L'obiettivo di questa tappa era quello di implementare la stecca e la sua gestione per il tiro. La stecca, rappresentata come parallelepipedo utilizzando la stessa mesh cube del Laboratorio 7, è soggetta a due traslazioni: la prima che serve per posizionarla nella posizione giusta, applicata subito dopo la scalatura, e la seconda per posizionarla correttamente rispetto alla posizione della biglia bianca, applicata dopo le rotazioni. La rotazione attorno all'asse x presenta un piccolo offset in modo che l'osservatore non si trovi all'interno della stecca al momento del tiro. La stecca inoltre viene disegnata solo in modalità AIM. Per gestire la posizione durante il caricamento del tiro una handle apposita rileva il movimento mentre il tasto sinistro è premuto e aggiorna il valore cue_displacement, utilizzato nella prima traslazione alla quale è soggetta la stecca. Al momento del rilascio il valore del tiro viene calcolato da calculateShot, dall'angolo si ricavano le due componenti di direzione, ed entrambe vengono moltiplicate per la forza del tiro, proporzionato in una scala da 0 a max_speed applicando le due componenti ottenute alla biglia bianca.

Tappa 9

L'obiettivo era di introdurre le buche e la logica ad esse associate. Le buche sono state introdotte scartando i frammenti delle sponde e del panno. Sono stati definiti 6 punti rappresentanti i centri delle buche e viene scartato ogni frammento nel raggio della buca. Successivamente è stato aggiunto un rivestimento interno come cilindro con culling face front in modo che si potesse visualizzare correttamente solo le facce interne. Inoltre vengono scartati i frammenti che facevano in modo di vedere parte del cilindro anche guardando il campo da dietro la buca. Successivamente è stato implementato il rilevamento della buca testando la distanza tra il centro della sfera e il centro di ogni buca. Se quest'ultima è minore del raggio della buca la biglia procede verso il centro della buca fino a fermarsi quando le due combaciano. A questo punto la biglia sprofonda per l'altezza del tavolo e lo stato diventa POTTED. Tutte le biglie con stato POTTED non vengono disegnate nella draw di Scene.

Inoltre è stata corretta la forma della stecca da parallelepipedo a cilindro. Per disattivare le sponde in corrispondenza delle buche è stato introdotto un flag `near_pocket` che, se attivato, non fa andare a buon fine la chiamata di `detect_rail_collisions()`.

È stata anche introdotta la logica di riposizionamento della biglia bianca se essa viene imbucata.

Tappa 10

L'obiettivo era di implementare il sistema di regole e di giocatori in modo da poter effettivamente giocare una partita regolare. Le regole implementate sono:

- imbucamento della biglia bianca (già presente nella tappa 9 ma refactorizzata e inserita nella nuova classe `GameState` che gestisce la logica delle regole)
- assegnamento dei gruppi delle biglie: il gruppo al quale la prima biglia che viene imbucata appartiene quando entrambi i giocatori non hanno

già un gruppo assegnato diventa il nuovo gruppo assegnato al giocatore di quel turno, all'altro giocatore viene assegnato l'altro.

-sistema di turni: i turni vengono riconosciuti dal colore della stecca:

beige giocatore 1

marrone scuro giocatore 2

-falli:

- imbucamento della biglia bianca: riposizionamento nella posizione originale e passaggio del turno
- colpire come prima biglia una biglia non del proprio gruppo: passaggio del turno
- non colpire alcuna biglia: passaggio del turno
- colpire come prima biglia la numero 8 quando si ha un gruppo assegnato

-passaggio turno: il turno viene passato se non si imbucava una biglia del proprio gruppo mentre viene mantenuto altrimenti

Condizioni di vittoria e sconfitta:

Vittoria:

- Imbucata la biglia numero 8 al primo tiro della partita (possibile solo per giocatore 1)
- Imbucata la biglia numero 8 dopo aver imbucato tutte le biglie del proprio gruppo

Sconfitta:

- Imbucare la biglia 8 prima di aver imbucato tutte le biglie del proprio gruppo
- Imbucare la biglia 8 commettendo fallo nello stesso turno (imbucando anche la biglia bianca o non toccando la biglia 8 come prima biglia)

Difficoltà riscontrata: la funzione `resolve_shot()` nella nuova classe che gestisce la logica del gioco `GameState` veniva chiamata ad ogni frame dopo la fine del tiro annullando la logica al suo interno.

Soluzione: introdotto un flag `shot_in_progress` che viene settato a `true` non appena viene effettuato il tiro e a `false` non appena la funzione `resolve_shot()` viene chiamata nel main loop. La funzione `resolve_shot()` viene chiamata solo se non è rilevato movimento e se il flag è attivo.

Tappa 11

L'obiettivo principale di questa tappa era quello di rifinire il gioco in alcune sue parti.

È stato introdotto l'audio includendo SFML/Audio.hpp per i rumori:

- stecca che colpisce la biglia bianca
- biglia che urta un'altra biglia
- biglia che colpisce una sponda
- biglia che viene imbucata

Fonti dei suoni: <https://freesound.org/>

Successivamente ho cambiato il meccanismo di riposizionamento della biglia bianca: ogni volta che si commette fallo la biglia bianca viene riposizionata al centro del campo e un flag `cue_to_reset` viene attivato. Questo flag rende impossibile passare in modalità AIM finché la biglia non è stata riposizionata. La biglia bianca può essere mossa tramite le freccette per poi confermarne la posizione premendo ENTER. Quando viene premuto il tasto ENTER si controlla la posizione della biglia bianca: se essa è compenetrata con un'altra biglia o è troppo vicina ad una buca, la pressione del tasto viene annullata.

Possibili sviluppi futuri:

- Evitare la compenetrazione della stecca con le biglie e con le sponde
- Schermate di vittoria e menu integrate nel programma
- Possibilità che lo spin influenzi traiettoria e urti delle biglie